zBeyond Chunk-level RAG :Hierarchial Document-Aware Retrieval for Indian Legal Judgements

S Tejeswar
Dept., of Artificial Intelligence and Data Science
Chennai Institute of Technology
Chennai, India

A Arivarasan
Dept., of Artificial Intelligence and Data Science
Chennai Institute of Technology
Chennai, India

Ms.Shunmuga Sundari
Dept., of Artificial Intelligence and Data Science
Chennai Institute of Technology
Chennai, India

***Abstract*— Legal texts are known to contain complex legal jargon, dense reasoning, and document structure, which may be hard to comprehend. Due to this, conventional Retrieval Augmented Generation (RAG) techniques could not produce optimal results on legal text on various benchmarks. Legal queries often require the correct document for context, rather than relevant chunks across various documents, which plain RAG struggles with. In this paper, we produce a new hybrid document retrieval technique, which significantly improves the retrieval quality for legal judgments. The system consists of extracting legal judgments of the Supreme Court of India and generating concise, semantically rich summaries for each judgment to capture legal reasoning and important facts. Parallelly, each judgement is partitioned into coarse-grained chunks (larger-sized chunks) to preserve detailed factual and doctrinal nuances. Both the case summary and coarse chunks are ingested to separate vector databases. For a given query, the system retrieves candidate documents based on both summary-level similarity and chunk-level similarity. For a given document, the chunk-level score is the maximum similarity score of all its corresponding chunks retrieved. A final document relevancy score is computed as a weighted combination of the summary-level similarity score and the chunk-level score. The selected documents are partitioned into fine chunks (smaller chunks), which are passed to a reranking mechanism. Reranked fine chunks are used as context for answer generation.**

Keywords—Legal Document Analysis, Retrieval Augmented Generation, Hierarchical retrieval, Document retrieval, Semantic Similarity, Vector Search, Natural Language Processing.

I. INTRODUCTION

A fundamental requirement for legal research, case argument analysis, case law analysis is the retrieval of correct legal information. Indian Courts have a significant record of pending cases across courts of all levels. As per National Judiciary Data Grid, Govt. of India, as of January 11, 2026, about 92245 cases in Supreme Court of India, about 63,76,199 cases in High Courts and about 4,76,10,433 cases in District Courts are pending. One of the primary reasons of pending cases, is the tedious process of legal research as manual analysis of cases is very time taking and exhaustive.

Large-scale database of legal documents, including statutes, judgments, and orders, are now easily accessible to the public, due to the growing digitization of court records. Despite these advancements, the complexity of legal documents, legal terminologies and scattered nature of information across timeline, challenge legal information retrieval.

Retrieval-Augmented Generation (RAG) has become a promising paradigm for improving the factual foundation of large language models. By combining dense retrieval with generative models, RAG systems have proven to perform well in general-purpose domains. But applying traditional RAG methods directly to legal texts frequently yields less-than-ideal outcomes due to intricacies presented by legal text.

Most traditional RAG systems split the document into short passages and directly retrieve them based on the similarity with the query. A drawback of such a single-stage retrieval process is that they tend to retrieve seemingly relevant chunks, but they may not contain the correct context for answering the question [1]. Since most legal documents have a similar structure and have similar passages, retrieval of the correct passage becomes challenging.

When it comes to Indian court rulings, these difficulties are especially noticeable [2]. The legal documents issued by the Supreme Court of India contains incredibly lengthy and includes, intricate facts, procedural histories, several arguments, citations to earlier decisions, and constitutional interpretations such as various acts. The similarity of passages and interleaved and layered nature of reasoning phrases in legal documents often leads to retrieval of seemingly relevant information, rather than current needed information, which may distract the overall reasoning.

In this paper, we propose a hierarchical document-level retrieval system that combines document-level summarization with coarse-grained text representation, in an attempt to improve semantic document-level retrieval. This mechanism addresses the very need of legal queries—getting the correct information to answer and reason the question, rather than retrieving seemingly relevant chunks, thus improving legal reasoning. This makes the system highly appropriate for retrieving correct information from a large-scale dataset in Indian law and improve the reasoning of question answering.

II. LITERATURE SURVEY

A. LEGAL INFORMATION RETRIEVAL

With the advancement in language modeling and information retrievals, several interesting and useful works have been produced in legal information retrieval. Legal information retrieval has shifted from keyword matching toward dense vector representations.

For instance, an LSTM-based domain classification method has been implemented to categorize legal queries into sub-domains, effectively reducing search space and latency in large-scale legal repositories [3]. A. Garlapati *et al.* [4], describe a legal chatbot designed to support and answer legal queries regarding Indian law through the use of Legal BERT, GPT-2, and Retrieval-Augmented Generation, which serves as solid work for legal information retrieval.

To address the limitations of single-stage retrieval, Huynh *et al.* [5] proposed a hybrid retrieval architecture with local retrieval module with category classifier, which selects relevant legal domain, to perform specific retrieval and a global retrieval module combining both lexical and semantic retrieval methods. The results of both modules are fused to get final output.

To take legal information a step forward, Singh *et al.* [6] integrated NLP techniques for both document retrieval and case classification and also incorporates dense vectors and TD-IDF vectors to rank the documents based on query similarity. This can ensure both semantic similarity and keyword matching, which is very essential in legal text.

B. RETRIEVAL-AUGMENTED GENERATION FRAMEWORKS.

Retrieval-Augmented Generation (RAG), has emerged as a solution to solve the hallucination and the lack of external-knowledge problem for LLMs. Recent advancements in RAG focus on optimizing the synergy between the retriever and the generator to ensure contextual accuracy.

The RAPTOR model [7] introduces a framework for hierarchical retrieval that is suitable for long documents and follows the strategy of clustering and abstractive summary generation at various levels. The strategy of retrieving at a tree structure helps to perform both coarse and fine searches at the semantic level.

IRAGKR [8] presents an iterative retrieval approach for knowledge that iteratively enhances knowledge retrieval by performing multiple knowledge retrieval-generation passes to filter out unnecessary information gradually. Unlike conventional RAG, this method fixes the noise buildup that is collectively accumulated within an information pipeline. MS-RAG [9] proposes a retrieval method, that uses multiple semantic representations, capturing different aspects of meaning within documents. Further the retrieval accuracy is improved with semantic fusion and reranking. Experimental results suggests that multi-semantic representation, enhances retrieval quality compared to single-embedding approaches.

HyPA-RAG [10] presents a hybrid retrieval method with dense vector retrieval combined with sparse lexical matching, with an adaptive reranking method. This helps in processing legal as well as policy documents. This approach has dynamic retrieval strategy which improves precision for complex environments, depending on the user query.

The challenge of navigating complex, standardized financial reports similar to legal text is addressed in [11], where a hierarchical retrieval method combined with evidence curation improves document retrieval and reasoning for complex queries. An iterative multi-hop framework with answer checker, ensures necessary correct context is retrieved to answer the answer accurately.

C. HIERARCHICAL AND MULTI-STAGE RETRIEVAL MODELS

Multi-stage hierarchical retriever frameworks consistently improve the retrieval of relevant and accurate passages compared to single-step RAG. These hierarchical methods perform information retrieval in multiple steps, by retrieving high level representation and gradually narrowing to fine passages, which significantly improves retrieval performance. Some of these frameworks are:

The AQgR framework [12] introduces a specialized question-guided retrieval approach for the Indian Case Judgements question-answering. By integrating structured template-based summaries for judgement retrieval and breaking down user query to multiple short questions, the system effectively navigates the complexities of Indian case law to provide more precise legal citations [12].

Beyond static retrieval, *Hudson de Martim* [13] introduced a Graph RAG approach specifically tailored for complex legal text. Their framework possesses an innovative hierarchical and temporal structure, allowing the system to handle the evolution of legal rules over time while maintaining a deterministic retrieval path to ensure consistency in judicial applications [13]

III. METHODOLOGY

This section covers all the important points in its implementation of the hybrid document retriever with the following sub-sections

A. DATA COLLECTION AND PREPARATION

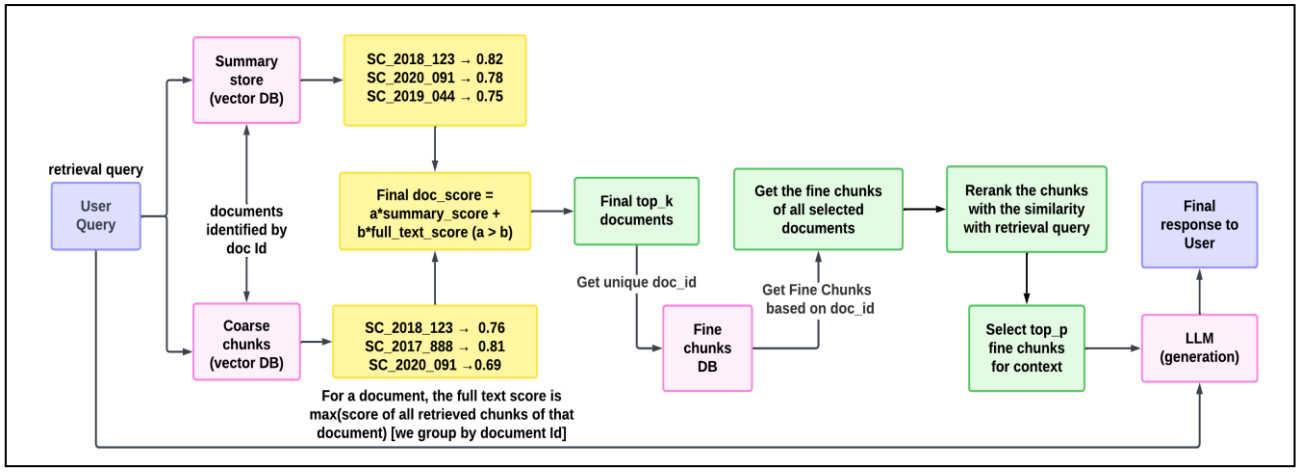
The system uses the case judgements of the Supreme Court of India. The judgments are openly accessible from the following link [14]. We extracted the judgments from an openly accessible AWS S3 bucket [15], where all Supreme Court cases from 1950 to 2025 are downloaded and stored. The bucket also stores raw metadata for each judgement, such as petitioner(s), respondent(s), judge(s), CNR number, citations, decision date, disposal nature, neutral citation, year, etc. This metadata provides useful information about cases and can help to improve the retrieval performance.

The case judgement contains a lot of noise (header, footer, etc) and had to be cleaned before actual utilization. For data cleaning and preprocessing, we checked common terms that are present in every page and meticulously removed them, to make text continuous.

B. DOCUMENT SUMMARIZATION

The primary aim of document summarization involves capturing the global legal context of the case and generating a dense and semantically rich extract, which represents all important aspects and facts. The summary must enclose all necessary details, which are helpful in accurate retrieval of the case document from a vast corpus.

As a part of LLM for generating summaries, we utilized the Llama 3.2 3 billion parameter model for summarization of documents. It is openly available and provides significant performance across comparable models of its size. It's a general-purpose language model, with a good understanding of complex legal terminologies. Its impressive 131072-token context length makes it a viable option for summarization of long documents.



(Figure 1) Architecture of the Hybrid Legal Retriever

The text in each case judgement PDF is extracted and combined with the summarization prompt. The summarization prompt is so structured to extract important fields from the judgement, such as petitioner, respondents, background of the case, arguments of petitioner and respondents, various issues discussed in court, final order, the reasoning of the court for the order, important precedents cited, and list of acts. This helps to extract the core issue of the case and the reason for the judgement.

The generated summary for each document is stored with a unique document ID, which helps to identify the document and aggregate the retrieved coarse chunks based on the source document.

C. CHUNKING STRATEGY

The next step in the implementation is to partition the extracted text of the judgment into coarse chunks (larger-sized chunks). The intention of coarse chunking is to preserve the subtle legal nuances of the judgement, which may be crucial for document retrieval, which may be buried or ignored in document summaries. Combining the coarse chunks with judgement summary helps to ensure that the overall case context as well as subtle nuances and facts are taken into account during retrieval.

Splitting of case judgement text into chunks, based on a fixed chunk size, does not preserve the semantic meaning of the chunks, especially in the case of legal judgements. It leads to a loss of information and harms the retrieval performance. We decided to follow a structure-preserving chunking approach—we first find the various sections of the document and recursively split each section into coarse chunks. This enables us to chunk based on the structure of the document and also based on the semantic meaning.

The sections in the document text are identified based on keywords such as "judgement/order," "headnotes," "citations," "acts," etc. This avoids splitting across sections, which may lead to loss of structural information and semantic relationships within a chunk. Regular expressions were used to identify various splitting terms, and various sections were stored. It is to be noted that while portioning the judgement text into different sections, very strict splitting rules may lead to splitting of text within a section. Thus, after reviewing several case judgements, the splitting expressions were developed.

We utilized RecursiveCharacterTextSplitter from the LangChain library to split each section into semantically meaningful chunks. For coarse chunking, we set the chunk_size (maximum size of chunk based on number of characters) as 2000 characters and chunk_overlap as 200 characters. The chunked data is stored for ingestion into a vector database. For fine chunking, the chunk_size is 1000 with chunk_overlap as 150.

For each document, a unique document ID is given during the document summarization phase and stored in a CSV file. During coarse chunking of the document, it is necessary that each chunk of the document should be linked to the corresponding document through document ID. This helps to aggregate the retrieved chunks based on document ID , during retrieval of chunks. Thus, each chunk has a metadata or payload field, where both the chunk ID and document ID are stored.

D. EMBEDDING AND VECTOR STORAGE

The coarse chunks and the generated document summary are converted to respective text embedding vectors for vector storage. We employed the text embedding model BAAI/bge-large-en-v1.5 from Hugging Face. BAAI/bge-large-en-v1.5 is a high-performance English text embedding model, developed by Beijing Academy of Artificial Intelligence (BAAI), which is optimized for retrieval, clustering semantic similarity, and classification tasks. It uses a BERT (large) architecture as a base and has a size of about 1.34 GB. It produces text embedding vectors of 1024 dimensions, which helps to represent the complex semantics in legal documents and helps in efficient retrieval.

For storing the embedding vectors in a vector store, we employed the Qdrant vector database. Qdrant is an open-source vector database and similarity search engine designed for high-performance AI applications. It provides low latency and high throughput even with billions of high-dimensional vectors.

Qdrant also has the hybrid retrieval capability, which helps to combine dense vectors (for semantic meaning) and sparse vectors (for keyword matching) for improving retrieval accuracy. Its payload indexing and filtering option, which helps to conditionally retrieve vectors based on the metadata (payload), helps retrieve legal documents based on metadata fields (e.g., based on petitioner name or case type).

The document summaries and the course chunks are ingested in separate Qdrant collections. The coarse chunks are linked to summaries by document ID.

E. HYBRID DOCUMENT RETRIEVAL MECHANISM

The pseudocode for the hybrid document retrieval process is described. For a given user question, the question is converted to vector embedding, using the same embedding model used while ingesting documents. The query is used to retrieve the top K relevant summaries from the summary vector store. Parallelly, the query is used to retrieve top Kc coarse chunks from the coarse vector store.

The retrieved summaries have a single relevancy score, called the summary score. The retrieved chunks can be from more than one document. Thus, we aggregate the chunks based on the document ID , and define the coarse chunk score of a document as the maximum similarity score of all coarse chunks of that particular document retrieved.

For a retrieved document (either through summary or coarse chunk), the final document score is computed as the weighted combination of the summary score and the coarse chunk score, where more weight is given to the summary score:

$$Score(d)=\alpha \cdot S_summary(d)+\beta \cdot max(S_chunk(d))$$

$$(where,\ \alpha>\beta)$$

where $S_summary\ (d)$denotes the similarity score between the query and the document summary, and $S_chunk\ (d)$represents the similarity scores of coarse chunks associated with document d. The weighting constraint $\alpha > \beta$ensures that document-level semantic coherence remains the primary retrieval signal, while chunk-level evidence contributes supplementary fine-grained relevance. Finally, documents are ranked based on their aggregated scores, and the top-P documents are returned as the retrieval output.

Fine chunks from the final selected document are selected using the document id. These fine chunks are reranked using a suitable re-ranker, to drill down to the passages, that has the answer to the query, and neglect unwanted passages, which may distract the reasoning. Llama 3.2 3B model is used as the re-ranker model for the fine chunks.

The re-ranked passage, along with the query is sent to the LLM to generate the response. We again use the Llama 3.2 3B as the response generating LLM. This hierarchical and hybrid retrieval method enables the system to balance the global legal reasoning captured in document summaries and the factual details and nuances present in the text.

ALGORITHM 1: HYBRID DOCUMENT RETRIEVAL

INPUT: User Query Q, Number of top coarse to retrieve K_c, Number of top document summaries to retrieve K_s, final top documents K_p, Summary store S, Coarse chunk store C

OUTPUT: Ranked list of relevant documents with supporting chunks, R

```
Encode query Q into embedding q
Retrieve top-Ks summaries from S using similarity search with q
Retrieve top-Kc coarse chunks from C using similarity search with q
Set D = set of all document id retrieved from S and C
For each retrieved summary di in D do
    Compute max chunk similarity score sc(di)
    Let ss(di) be the summary similarity score
    Compute final score:
        Score(di) = α · ss(di) + β · sc(di)
end for
Rank all documents di in descending order of Score(di)
For each ranked document di do
    Attach its supporting chunks
end for
Return top Kp documents in ranked list R
```

IV. RESULTS AND DISCUSSION

This section presents the empirical evaluation of the proposed Hybrid Summary-Chunk Retrieval mechanism against the Coarse Chunk Retrieval approach on the internal benchmark of Supreme Court of India judgments.

The internal benchmarking dataset consists of a manually curated set of queries where each query was associated with a set of relevant documents (through document ID) and gold evidence chunks. Both the retrieval methods were evaluated under identical conditions.

Method	**Recall @ k (%)**		**Precision @ k (%)**	
	k = 5	k = 10	k = 5	k = 10
Coarse Chunk Retriever	32.23	47.52	62.72	48.44
Hybrid Retriever	53.64	67.24	65.14	49.23

Table1: Comparison of document level Recall and Precision

Method	**MRR**	**nDCG@5**	**nDCG@10**
Coarse Chunk Retriever	0.87	62.72	48.44
Hybrid Retriever	0.92	66.14	52.23

Table 2: Comparison of document level MRR and nDCG

Table 1 compares document level Recall@k and Precision@k for both methods. At k = 5, the Coarse Chunk Retriever achieves a Recall@5 of 32.23%, whereas the Hybrid Retriever attains a higher Recall@5 of 53.64%. This provides that incorporating document-level summaries significantly improves the ability to retrieve judgements based on global context which serves as context for queries across multiple judgements.

A similar trend is observed for recall@10, which increases from 47.52% for the Coarse Chunk method to 67.24% for the Hybrid approach. The increase in the Recall score from k = 5 and k = 10 can be attributed to a larger retriever set (k = 10) that allows to incorporate more relevant documents.

The Coarse Chunk Retriever records a Precision@5 of 62.72%, which drops to 48.44% at k = 10. The Hybrid Retriever shows a slightly higher Precision@5 of 65.14% and a comparable Precision@10 of 49.23%. The drop in precision as k increases can be related to inclusion of lower-ranked and

less relevant documents. The improvement in precision by the Hybrid approach suggests reduce in false positive error and the ability to identify correct document as relevant.

Table 2 compares the ranking quality of both methods with Mean Reciprocal Ranking (MRR) and Normalized Discounted Cumulative Gain (nDCG) at k = 5 and 10. In this work, nDCG is computed under a binary relevance setting, where each retrieved document is assigned a relevance score of 1 if it belongs to the ground-truth relevant set and 0 otherwise. The Discounted Cumulative Gain (DCG@k) is calculated by summing the discounted relevance of the top-k ranked documents, and the Ideal DCG (IDCG@k) is obtained by placing all relevant documents in the top positions.

The higher MRR score by the hybrid retriever (0.92) than coarse chunk retriever (0.87) demonstrates, hybrid retriever places the first relevant document closer to the top position which also explains improved precision. Higher MRR is particularly needed for legal search engines, as users typically inspect the top retrieved documents.

The Hybrid method has better nDCG scores, obtaining nDCG@5 of 66.14% and nDCG@10 of 52.23%, compared to 62.72% and 48.44% respectively for the Coarse Chunk Retriever. The higher nDCG values demonstrate that relevant documents are retrieved and ranked closer to the positions of ideal ordering. The consistent performance suggests that document summaries, helped to retrieve relevant documents based on semantic meaning, and place at top positions.

The experimental results demonstrate that the proposed Hybrid Retrieval framework consistently outperforms the Coarse Chunk baseline in retrieving correct related document. These findings suggest that integrating semantic case summaries with coarse chunks, ensures context-aware and factual retrieval, which is crucial for complex legal research.

V. CONCLUSION

In conclusion, the proposed hierarchical legal document retrieval framework serves as a solution, for processing complex legal text and improving the retrieval of documents, through semantic and factual relevancy. This addresses the limitations of single-step RAG systems, through hierarchical retrieval strategy. Experimental results on internal dataset of Indian Supreme Court judgments show that the proposed method outperforms the coarse-chunk retrieval method on various metrics—recall, precision, MRR, and nDCG. By jointly capturing the global context of judgements and factual legal nuances by coarse chunks, the proposed method attempts to achieve better context-aware and accurate document retrieval. This helps to retrieve correct fine-grained chunks, rather than seemingly relevant chunks, which aids in correct reasoning for queries. These findings highlight the importance of hierarchical semantic representations in legal systems for more accurate document retrieval.

VI. REFERENCES

[1] M. G. Arivazhagan *et al.*, "Hybrid hierarchical retrieval for open-domain question answering," in *Findings of the Association for Computational Linguistics: ACL 2023*, Toronto, Canada, July 2023, pp. 10680–10689.

[2] A. Sharma, Akash, A. Singh, D. Shrivastava and Pawan, "Hybrid Legal Language Simplification for Indian Judiciary: Leveraging LegalBERT and Summarization Techniques," *2025 International Conference on Next Generation Information System Engineering (NGISE)*, Ghaziabad, Delhi (NCR), India, 2025, pp. 1-4, doi: 10.1109/NGISE64126.2025.11085325.

[3] R. R. Dhala, A. V. S. P. Kumar and S. P. Panda, "Legal Document Information Retrieval using Long Short Term Memory-Based Domain Classification Technique," *2025 International Conference on Innovations in Intelligent Systems: Advancements in Computing, Communication, and Cybersecurity (ISAC3)*, Bhubaneswar, India, 2025, pp. 1-6, doi: 10.1109/ISAC364032.2025.11156333.

[4] A. Garlapati, H. Koutharapu and N. Doddi, "Enhancing Public Access to Legal Knowledge in India: A Legal Chatbot Using Legal BERT, GPT-2, and Retrieval-Augmented Generation (RAG)," *2025 IEEE International Conference on Emerging Technologies and Applications (MPSec ICETA)*, Gwalior, India, 2025, pp. 1-6, doi: 10.1109/MPSecICETA64837.2025.11118538.

[5] H. M. Huynh, L. M. H. Nguyen, T. Q. Pham, K. H. Mai and K. Van Nguyen, "Local Focus, Global Coverage: A Hybrid Retrieval Architecture for Legal Text," *2025 17th International Conference on Knowledge and System Engineering (KSE)*, Da Lat, Vietnam, 2025, pp. 1-6, doi: 10.1109/KSE68178.2025.11309639.

[6] M. R. Singh, S. Ahmad, K. Sharma, P. Sharma and P. Kaushik, "AI-Driven Legal Assistance: Optimizing Document Retrieval and Case Classification with NLP Techniques," *2025 International Conference on Advances in Modern Age Technologies for Health and Engineering Science (AMATHE)*, Shivamogga, India, 2025, pp. 1-7, doi: 10.1109/AMATHE65477.2025.11080869.

[7] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," 2024, *arXiv:2401.18059*. [Online]. Available: https://arxiv.org/abs/2401.18059.

[8] K. Du *et al.*, "IRAGKR: Iterative retrieval augmented generation with fine-grained knowledge refinement," *Neurocomputing*, vol. 655, Art. no. 131282, 2025, doi: 10.1016/j.neucom.2025.131282.

[9] X. You, Y. Luo, and L. Gu, "MS-RAG: Simple and effective multi-semantic retrieval-augmented generation," in *Proc. 2025 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Suzhou, China, Nov. 2025, pp. 22609–22625, doi: 10.18653/v1/2025.emnlp-main.1151.

[10] Rishi Kalra *et al. "HyPA-RAG:* A hybrid parameter adaptive retrieval-augmented generation system for AI legal and policy applications," 2024, *arXiv:2409.09046*. [Online]. Available: https://arxiv.org/abs/2409.09046.

[11] Jaeyoung Choe *et al.*, "Hierarchical retrieval with evidence curation for open-domain financial question answering on standardized documents," 2025, arXiv:2505.20368. [Online]. Available: https://doi.org/10.48550/arXiv.2505.20368

[12] Vishnuprabha V et al., "Augmented question-guided retrieval (AQgR) of Indian case law with LLM, RAG, and structured summaries," 2024, arXiv:2508.04710. [Online]. Available: https://arxiv.org/abs/2508.04710

[13] Hudson de Martim, "Graph RAG for legal norms: A hierarchical, temporal and deterministic approach," 2025, *arXiv:2505.00039v2*. [Online]. Available: https://arxiv.org/html/arXiv:2505.00039v2.

[14] e-Committee, Supreme Court of India, "Judgments & Orders," *eCourts Services*. [Online]. Available: https://judgments.ecourts.gov.in/.

[15] Indian Supreme Court Judgments was accessed on 30th December 2025 from https://registry.opendata.aws/indian-supreme-court-judgments.